CS 639 Homework 2: From Planning to Learning

- **Due** Wednesday, October 21, on Gradescope (the exact time is there); hard close 48 hours later. Slip days (four shared by HW1–HW4, at most two here) are applied automatically; late time beyond them costs 10% of the maximum score per started 24-hour period.
- **Weight.** 10% of the course grade; 100 points: Problems 1–4 carry 28, 22, 20, 30. Problems 3–4 are the programming part, 50 points, graded from the five figures in your report. Most students should expect about three hours. The Assistance Statement at the end is required.
- **What to submit.** One file: `hw2.pdf`, typeset in LaTeX from the template `hw2.tex` (handwritten or scanned work is not accepted). Problems 1 and 2 are solved by hand: under each part, write the equations and the computation that lead to your answer, then the answer; the template marks the place, and its `solution` environment typesets your work in blue. Problems 3 and 4 are the five figures `hw2.py` saves, each included under its part; nothing the script prints goes into the report, and `hw2.py` itself is not submitted. No part asks for an explanation in words.
- **Grading.** In Problems 1 and 2 the derivation carries the credit: a correct method with an arithmetic slip loses little, a bare number without its derivation earns at most half, and a part that reuses a slipped number from an earlier part is graded on its own logic. In Problems 3 and 4 the figures are graded as they stand.
- **Collaboration and AI.** Discuss ideas with at most two named classmates; write every derivation, every line of code, and every answer yourself. Problems 1(c) and 2(a) are individual derivation checks (**Red**: no AI). Everything else is **Yellow**: limited, disclosed use, never for the computations or the code you are asked to produce. List every collaborator, resource, and tool in the Assistance Statement.
- **What you need.** Lectures 5–9 and Sutton and Barto, Sections 4.2–4.7, 6.1–6.3 (Example 6.2 is the random walk of Problems 3–4), and 8.1. Each problem states the definitions and results it uses; `hw2.py` contains the planning functions Problems 3–4 build on. Start with Problem 1 and the code; ask on Piazza early.

Notation.

- An MDP is $(\mathcal{S}, \mathcal{A}, P, r, \gamma, d_0)$ with finite $\mathcal{S}$ and $\mathcal{A}$, $r(s,a) = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a]$, and $\gamma \in [0,1)$ unless said otherwise; terminal states are absorbing with reward 0; $R_{\max}$ bounds $|R_{t+1}|$. A deterministic policy π leaves the chain $P^\pi(s' \mid s) = P(s' \mid s, \pi(s))$, $r^\pi(s) = r(s, \pi(s))$, and its exact value is the linear solve $V^\pi = (I - \gamma P^\pi)^{-1} r^\pi$. $V^\star, Q^\star$ are the optimal values and $\pi^\star$ an optimal policy.
- For $V \in \mathbb{R}^{|\mathcal{S}|}$, the lookahead table and the two Bellman operators are

$$Q_V(s,a) \coloneqq r(s,a) + \gamma \sum_{s'} P(s' \mid s,a)\, V(s'),$$

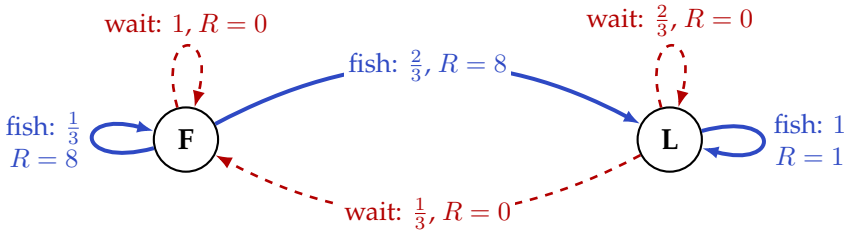
$$(\mathcal{T}^\pi V)(s) \coloneqq Q_V\big(s, \pi(s)\big) = r^\pi(s) + \gamma \sum_{s'} P^\pi(s' \mid s)\, V(s'), \qquad (\mathcal{T} V)(s) \coloneqq \max_a Q_V(s,a),$$

so $V^\pi = \mathcal{T}^\pi V^\pi$, $V^\star = \mathcal{T} V^\star$, $Q_{V^\pi} = Q^\pi$, $Q_{V^\star} = Q^\star$.

- A deterministic π is *greedy with respect to* V if $\pi(s) \in \arg\max_a Q_V(s,a)$ for every s; then $\mathcal{T}^\pi V = \mathcal{T} V$, and $\mathcal{T}^\pi V \le \mathcal{T} V$ for every π (inequalities between vectors hold entry by entry). The norm is $\|V\|_\infty = \max_s |V(s)|$; both operators are γ-contractions in it, and $V = \mathcal{T} V$ and $V = \mathcal{T}^\pi V$ have exactly one solution each, $V^\star$ and V^π (Lectures 3 and 5).
- Policy iteration (Lecture 6) evaluates π_k exactly and improves greedily with the *tie rule*, $\pi_{k+1}(s) \in \arg\max_a Q^{\pi_k}(s,a)$ with $\pi_k(s)$ kept whenever it is among the maximizers, until $\pi_{k+1} = \pi_k$. A *round* is one evaluation and one improvement; *evaluations* counts the linear solves, the policy stopped at included.
- The *loss* of a policy is $\|V^\star - V^\pi\|_\infty$; the advantages of π are $A^\pi(s,a) := Q^\pi(s,a) - V^\pi(s)$, and $\delta_k := \max_{s,a} A^{\pi_k}(s,a) = \|\mathcal{T} V^{\pi_k} - V^{\pi_k}\|_\infty$ is the residual of π_k (Lecture 7).
- An *iteration* is one application of $\mathcal{T}$ to a whole table (never in place); value iteration is $V_{k+1} = \mathcal{T} V_k$ from $V_0 = 0$, stopped at the first K with $\|V_K - V_{K-1}\|_\infty \le \theta$; $k_{\pi^\star}$ is the first k whose greedy policy of V_k is optimal.

1 A Pond: Policy Iteration by Hand [28 points]

A village fishes one pond. Each season the pond is either *full* (F) or *low* (L), and the village either *fishes* (f) or *waits* (w). Fishing a full pond pays 8 and leaves it full with probability $\frac{1}{3}$, low with probability $\frac{2}{3}$; fishing a low pond pays 1 and leaves it low. Waiting pays nothing: a full pond stays full, and a low pond recovers to full with probability $\frac{1}{3}$ and stays low otherwise. $\gamma = 0.9$. Write $(a_\mathrm{F}, a_\mathrm{L})$ for the deterministic policy that takes a_F when the pond is full and a_L when it is low. (`hw2.py` contains this model as `pond()`; Figure 1 of Problem 3 runs your `policy_iteration` on it.)



Solid: fish. Dashed: wait. Each arrow carries the probability of the transition and the reward R_{t+1} paid on it.

(a) **[12 points]** Run policy iteration by hand from $\pi_0 = (\mathrm{w}, \mathrm{w})$ with the tie rule, until the policy stops changing. In each round k:

- solve the two Bellman equations of π_k for V^{π_k};
- compute the four entries of $Q^{\pi_k}(s,a) = Q_{V^{\pi_k}}(s,a)$;
- read off the improved policy π_{k+1}.

Then report the number of evaluations, $V^\star$, and $\pi^\star$.

(b) **[8 points]** For each round k of (a), compute

- the loss $E_k = \|V^\star - V^{\pi_k}\|_\infty$;
- the ratio E_{k+1}/E_k for the rounds with $E_k > 0$, and verify that it is at most γ, as Lecture 7's theorem promises;
- the residual $\delta_k = \max_{s,a} A^{\pi_k}(s,a)$, the largest advantage;
- the loss the residual certifies, $\delta_k/(1-\gamma)$.

Then evaluate the fourth deterministic policy, (w, f), and compute

- the gap Δ, the smallest loss among the deterministic policies that are not optimal;
- the first round k at which Lecture 7's Corollary 2 guarantees $\pi_k = \pi^\star$, from E_0 alone.

(c) **[8 points]** **[Red]** Let π_k be any deterministic policy and π_{k+1} a greedy policy with respect to Q^{π_k}, ties broken in *any* way (not necessarily by the tie rule). Prove that a round which leaves the value unchanged has already reached an optimal policy:

$$V^{\pi_{k+1}} = V^{\pi_k} \quad \Longrightarrow \quad V^{\pi_k} = V^\star.$$

Conclude that if π_k is not optimal, then $V^{\pi_{k+1}}(s) > V^{\pi_k}(s)$ for at least one state s, whatever the tie-breaking.

Hint. You may use without proof that $V^{\pi_k} \le \mathcal{T}V^{\pi_k} \le V^{\pi_{k+1}}$ for every such π_{k+1}, however ties are broken, and that $V^\star$ is the unique solution of $V = \mathcal{T}V$.

Where this comes from: Lecture 6 (policy iteration, the tie rule, the improvement chain), Lecture 7 (the rate, the gap, the residual certificate), Lecture 3 (the linear solve); Sutton and Barto, Sections 4.2–4.3 and Exercise 4.4.

2 Certificates: What a Policy Read Off a Wrong Table Is Worth [22 points]

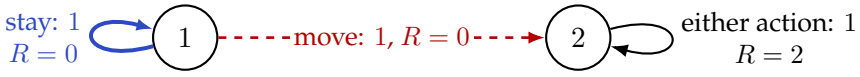
(a) **[12 points]** **[Red]** Let Q be any table on $\mathcal{S} \times \mathcal{A}$ with $|Q(s,a) - Q^\star(s,a)| \le \varepsilon$ for every pair, and let π be greedy with respect to Q, i.e. $\pi(s) \in \arg\max_a Q(s,a)$. Prove that

$$V^\pi(s) \;\ge\; V^\star(s) - \frac{2\varepsilon}{1-\gamma} \qquad \text{for every } s.$$

Hint. You may use without proof the identity

$$\begin{aligned} V^\star(s) - V^\pi(s) = &\big[Q^\star(s,\pi^\star(s)) - Q(s,\pi^\star(s))\big] \\ &+ \big[Q(s,\pi^\star(s)) - Q(s,\pi(s))\big] \\ &+ \big[Q(s,\pi(s)) - Q^\star(s,\pi(s))\big] \\ &+ \big[Q^\star(s,\pi(s)) - Q^\pi(s,\pi(s))\big]. \end{aligned}$$

(b) **[10 points]** Consider the two-state MDP below. In state 1, *stay* pays 0 and stays, *move* pays 0 and goes to state 2; in state 2 both actions pay 2 and stay; $\gamma = 0.9$. (`hw2.py` contains this model as `two_state()`, with stay as action 0.)



(i) Compute $V^\star$, $Q^\star$, and $\pi^\star$.

(ii) Let $\varepsilon = 1$ and $V = V^\star + \varepsilon\,(+1, -1)$, so that $\|V - V^\star\|_\infty = \varepsilon$. Compute Q_V and show that the two actions tie at state 1. Let π be the greedy policy that breaks the tie toward stay (as `greedy_policy` does); compute V^π and its loss $\|V^\star - V^\pi\|_\infty$.

(iii) Compute Lecture 7's bound $2\gamma\varepsilon/(1-\gamma)$ on this loss, and the bound of (a) with $\varepsilon_Q = \|Q_V - Q^\star\|_\infty$, the max over all pairs; show that the loss attains both.

Where this comes from: Lecture 7 (the greedy-policy bound and its Q form), Lecture 4 (greedy extraction and ties); Sutton and Barto, Section 4.4; Williams and Baird (1993) and Singh and Yee (1994).

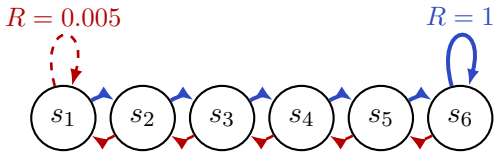
3 Algorithms in Code [20 points]

`hw2.py`, released with this handout, stores an MDP as the arrays `P[s, a, s2]` $= P(s_2 \mid s, a)$ and `r[s, a]` $= r(s, a)$, a deterministic policy as an integer array, and terminal states as absorbing rows with reward 0. Your task in this problem is to write the five functions marked `TODO`, in NumPy only; parts (a) and (b) say what each must do, and the docstrings are the specification. The file already contains:

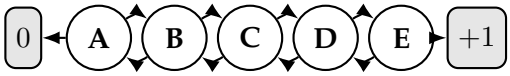
- the planning functions `policy_evaluation`, `q_from_v`, `greedy_policy` (ties go to the smallest action index), `value_iteration`, and `gridworld`, complete, with the helpers `induced_chain` and `optimality_test`;
- the models in the table below;
- `sample_counts`, Lecture 8's oracle: n samples at every pair, returned as counts and reward sums;
- `rw_episode`, one episode of the random walk as a list of transitions (s, r, s') in time order, $s' =$ `None` at the end.

The models, used here and in Problem 4:

Model	In `hw2.py`	γ	Defined	Used in
the pond	`pond()`	0.9	Problem 1	Figure 1
the two-state model	`two_state()`	0.9	Problem 2(b)	your own checks
the machine	`machine()`	0.9	Lectures 3–9	Figures 2 and 4
the Gridworld	`gridworld()`	0.9	Lecture 2 (below; step reward -0.04)	Figure 3
RiverSwim	`riverswim()`	0.95	below (weak current)	Figure 3
the random walk	`rw_episode()`	1	below (Sutton and Barto, Example 6.2)	Figures 2 and 5



RiverSwim (Strehl and Littman, 2008). Solid: swim right, against the current: from s_2 to s_5 it succeeds w.p. 0.3, stays put w.p. 0.6, and is pushed back w.p. 0.1; from s_1 it succeeds w.p. 0.3 and stays otherwise; from s_6 it stays w.p. 0.9 and is pushed back otherwise, earning 1 either way. Dashed: swim left, always one state left, earning 0.005 at s_1. All other rewards are 0.



The random walk: start at C, each step left or right w.p. $\frac{1}{2}$; the episode ends off either end, with reward 1 on the right and 0 elsewhere; $\gamma = 1$. True values $\frac{1}{6}, \ldots, \frac{5}{6}$.

The Gridworld is Lecture 2's: a 3×4 grid with one wall and two exits, four moves; a move goes where intended with probability 0.8 and slips to each side with probability 0.1, and into the wall or the border it stays put; every move costs the step reward -0.04, and entering an exit adds $+1$ or -1 and ends the episode.

Run `python hw2.py`. The script

- runs each of your functions on one small input and draws the results as Figures 1 and 2 (a panel whose function is not written yet says so);
- then runs the experiments of Problem 4, the same functions at scale, and saves Figures 3–5 (a block that needs an unwritten function is skipped, and so is its figure);
- prints every number behind the figures, for your own checking; nothing printed goes into the report.

(a) **[10 points] Write** the two planning functions, where marked `TODO`:

- `improve` (4 points), Lecture 6's improvement step with the tie rule: from a table Q and the current policy π, return the greedy policy of Q, keeping $\pi(s)$ wherever it is among the maximizers of $Q(s, \cdot)$.
- `policy_iteration` (6 points), Lecture 6's box: from a model and π_0, evaluate exactly, improve with `improve`, and repeat until the policy is unchanged; return the policies evaluated and their values.

Include Figure 1 (`hw2_fig1.png`) in your report; the script draws it from these two functions:

- left: `improve` on a 3×2 table Q with two near-ties, beside `greedy_policy`;
- right: policy iteration on the pond of Problem 1 from (w, w), the run of Problem 1(a): $V^{\pi_k}(s)$ against the round k, π_k under each round, $V^\star$ dashed.

(b) **[10 points] Write** the three learning functions, where marked `TODO`:

- `certainty_equivalence` (4 points), Lecture 8's box: from the counts and reward sums of `sample_counts`, estimate $\widehat{P}$ and $\hat{r}$ by averaging, then plan in the estimate with your `policy_iteration`; return the estimate, its optimal policy, and that policy's value.
- `td0_prediction` (3 points), Lecture 9's TD(0): over one episode, apply $V(s) \leftarrow V(s) + \alpha\,[r + \gamma V(s') - V(s)]$ to each transition (s, r, s') in time order, the terminal state worth 0; return the updated V.
- `mc_prediction` (3 points), Monte Carlo with the same template: over one complete episode, compute the return G_t that followed each transition, then apply $V(s_t) \leftarrow V(s_t) + \alpha\,[G_t - V(s_t)]$ for $t = 0, 1, \ldots$ in time order, every visit updated; return the updated V.

Include Figure 2 (`hw2_fig2.png`) in your report; the script draws it from these three functions:

- left: certainty equivalence on the machine from two samples per pair, your estimate and its plan over the true model;
- middle and right: TD(0) and Monte Carlo at $\alpha = 0.1$ from $V_0 = 0.5$, the five values after each of three fixed episodes of the random walk.

Where this comes from: Lecture 6 (the tie rule, policy iteration), Lecture 8 (certainty equivalence), Lecture 9 (the template and its two targets); Sutton and Barto, Sections 4.3, 6.1, 6.3, and Example 6.2. RiverSwim: Strehl and Littman (2008); the Gridworld: Russell and Norvig, Section 17.1.

4 From Planning to Learning, by Computer [30 points]

Nothing new to write here except the two step sizes of (c): `hw2.py` runs the three experiments below with your Problem 3 functions, on the models in its table, and saves one figure for each. Include each figure under its part.

(a) **[12 points]** The script runs policy iteration and value iteration ($\theta = 10^{-8}$) side by side, on the Gridworld from "always N" and on RiverSwim from "always left".

Include Figure 3 (`hw2_fig3.png`), the Gridworld left and RiverSwim right. Each panel shows, against k,

- value iteration's error $\|V^\star - V_k\|_\infty$, and the loss of the greedy policy of V_k;
- policy iteration's loss $\|V^\star - V^{\pi_k}\|_\infty$ round by round, with $\gamma^k \|V^\star - V^{\pi_0}\|_\infty$ dashed;
- dotted: $k_{\pi^\star}$, policy iteration's stop, and K.

(b) **[10 points]** The script runs your `certainty_equivalence` on the machine, planning in 1000 datasets of n samples per pair at each $n \in \{1, 2, 3, 5, 10, 20, 30, 60, 100\}$.

Include Figure 4 (`hw2_fig4.png`), both panels against n:

- left: the mean value error $\|\widehat{V} - V^\star\|_\infty$ and the mean loss of $\hat{\pi}$, with $21/\sqrt{n}$ dashed;
- right: the fraction of datasets with $\hat{\pi} \neq \pi^\star$.

(c) **[8 points]** The script runs TD(0) and Monte Carlo on the random walk from $V_0 = 0.5$, $\gamma = 1$, for 100 episodes, averaged over 100 runs: TD(0) at $\alpha \in \{0.05, 0.10, 0.15\}$, Monte Carlo at $\alpha \in \{0.01, 0.02, 0.03, 0.04\}$, and each method once more at a step size of your own.

Choose (2 points) your two step sizes, any value in $(0, 0.5]$ that the script does not already use for that method, and set them as `YOUR_ALPHA_TD` and `YOUR_ALPHA_MC` below the imports of `hw2.py`.

Include Figure 5 (`hw2_fig5.png`):

- left: the values of one TD(0) run after 0, 1, 10, 100 episodes, against the true values;
- right: the RMS error of every setting against the episode, yours marked in the legend.

Where this comes from: Lectures 6–7 (iterations against rounds, the rate), Lecture 8 (the sample size), Lecture 9 (the two targets, the step size); Sutton and Barto, Sections 4.3–4.4, 6.2, and 8.1.

Assistance Statement (required)

List every person and resource, including AI tools, that you used beyond the course materials, and for what: your collaborators (at most two) by name, any reference you consulted, and any AI tool with the parts it touched. Write "none" if none. You must be able to explain everything you submit.